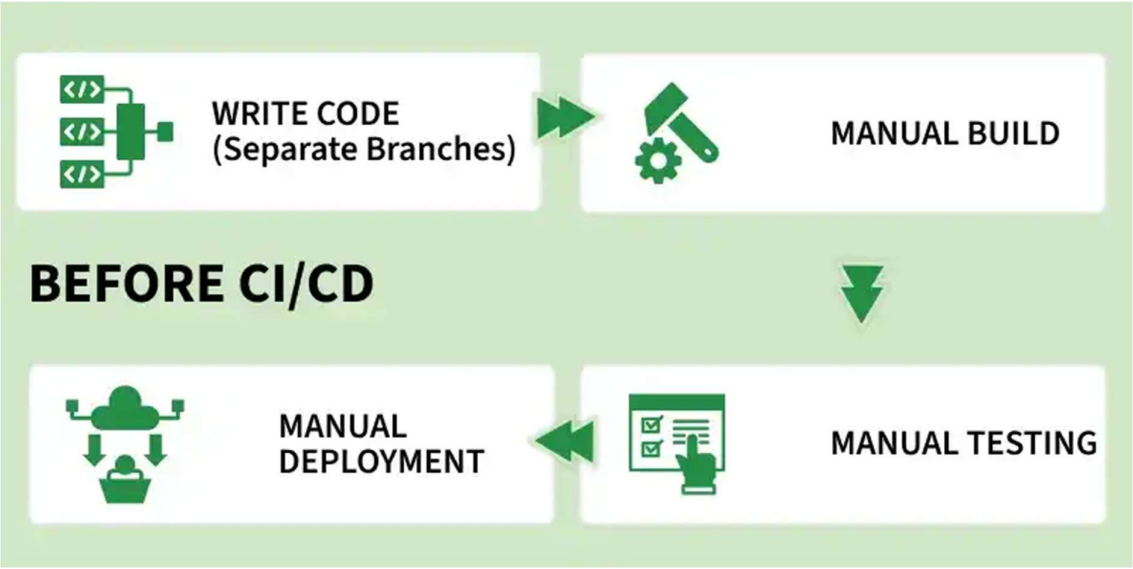
What is CI/CD?

CI/CD (Continuous Integration and Continuous Delivery/Deployment) is a modern software development practice that automates the process of building, testing, and releasing applications. It plays a key role in DevOps by streamlining collaboration between development and operations teams.

- Automates code integration, testing, and deployment workflows.
- Reduces manual effort while improving software quality.
- Enables faster, consistent, and reliable software releases.

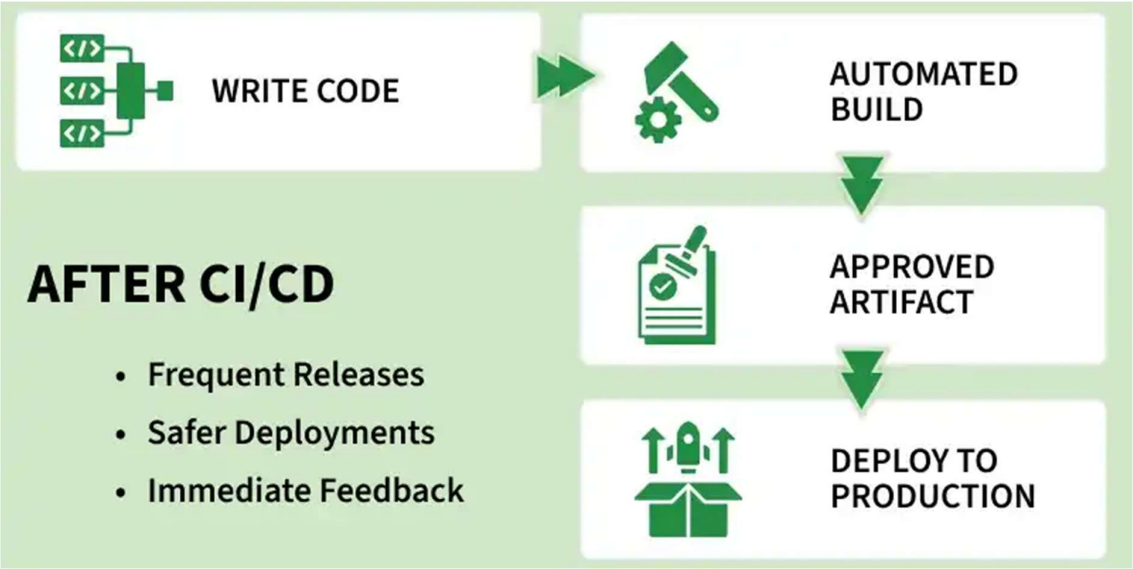
<u>Before CI/CD:</u>



- All branches were merged at the end, which often caused big conflicts and broken builds.
- Testing and building the code were also manual, usually done at the final stage, so bugs were discovered very late and were costly to fix.
- Deployment was a long and risky process, taking days or sometimes weeks, because everything was released together in one big update.

- Teams were also divided: developers wrote code, testers tested it, and operations managed servers.

<u>After CI/CD:</u>



- Developers commit code frequently to a shared repository.
- Automated pipelines run build and tests instantly.
- Bugs are detected early and fixed quickly.
- Continuous Delivery ensures code is always release-ready.
- Continuous Deployment enables automatic production releases.
- Smaller, frequent updates replace large risky releases.
- Improves collaboration and transparency across teams.

1. Continuous Integration (CI)

Continuous Integration focuses on integrating code changes frequently to avoid conflicts and ensure code stability.

- **Goal:** Prevent “integration hell” caused by late code merging.

- **Process:** Developers merge code changes into the main branch frequently (often daily).
- **Automation:** Each commit triggers an automated build and unit tests.
- **Outcome:** If tests fail, the build is rejected and developers are notified immediately.

2. Continuous Delivery (CD)

Continuous Delivery ensures that the application is always ready for release, with minimal manual effort.

- **Goal:** Keep the codebase in a release-ready state at all times.
- **Process:** After CI passes, code is deployed to a staging or testing environment.
- **Automation:** Integration, system, and performance tests are executed automatically.
- **Release:** Deployment to production is **manual**, typically triggered when needed.

3. Continuous Deployment (CD)

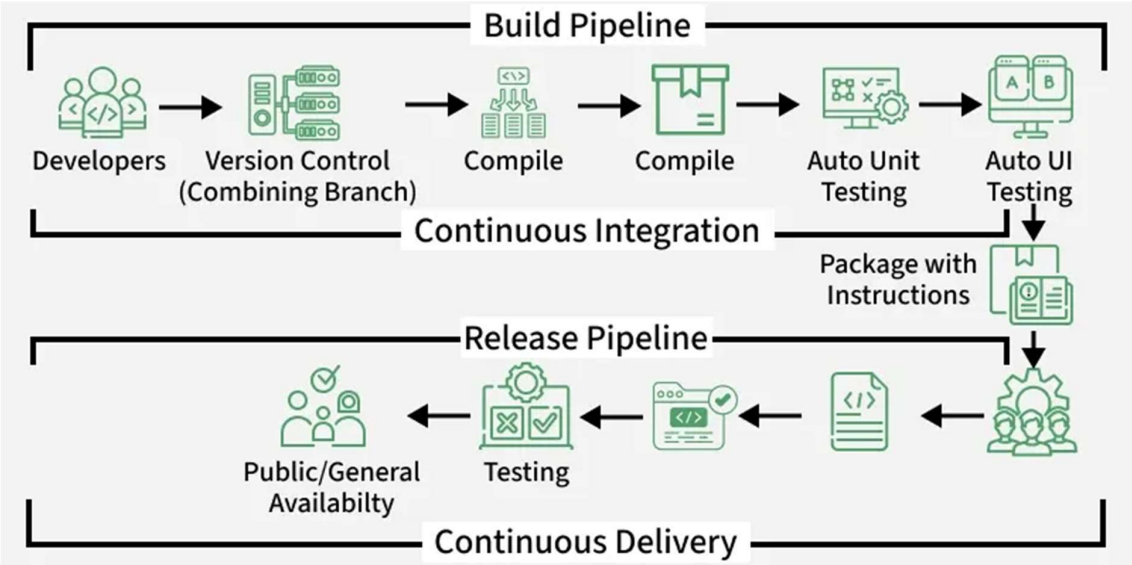
Continuous Deployment takes automation a step further by removing manual intervention in releases.

- **Goal:** Enable fully automated and faster production releases.
- **Process:** After all tests pass, code is automatically deployed to production.
- **Automation:** End-to-end pipeline runs without human involvement.
- **Requirement:** Requires highly reliable and comprehensive automated. testing.

CI/CD Pipeline:

A CI/CD pipeline is an automated workflow that enables teams to build, test, and deploy code efficiently and reliably.

In simple terms, it is a series of automated steps that help deliver software faster with fewer errors.



- Developers push code to version control.
- Code is compiled and built.
- Automated testing is performed.
- Application is packaged.
- Code moves through release pipeline.
- Delivered to users after validation.

How does an Application / Product go live?

Going live (deployment) is the process of making an application or product available to end-users, moving it from a development environment to a live

production environment. It involves rigorous testing, final deployment, and monitoring to ensure a smooth transition. It is the culmination of planning, development, and testing stages.

Key Steps to Go Live:

- **Final Testing & Quality Assurance:** User Acceptance Testing (UAT) is crucial, where stakeholders verify the app meets all requirements before launch.
- **Production Environment Setup:** Choosing a reliable hosting provider and setting up the infrastructure required for the live product.
- **Deployment (The "Go-Live"):** Moving the finalized code and content to the production server. This often involves enabling security features like Web Application Firewalls (WAF) and connecting custom domains.
- **Monitoring & Support:** Implementing real-time tracking of server health, site traffic, and user behaviour to identify and resolve issues instantly.
- **Post-Launch Review:** Reviewing the performance and gathering user feedback for future updates.

What are the different environments while working on a project?

1. Development (Dev)

This is where developers actively build features and fix bugs. It may run on a developer's local machine or a shared dev server. Data is usually dummy/sample, and changes happen frequently.

2. Testing / QA

This environment is used by testers to validate the application. QA checks functionality, UI, and integration flows. It is more stable than Dev, and bugs found here are reported back to developers.

3. UAT (User Acceptance Testing)

This environment is mainly for business users, clients, or product owners to verify that the system meets requirements. It is used to confirm workflows like checkout, login, reports, etc. Approval from UAT is often required before production release.

4. Staging / Pre-Production

This is almost identical to production. It is used for final validation such as deployment testing, performance checks, and end-to-end testing. The goal is to ensure everything works exactly as it will in production.

5. Production (Prod / Live)

This is the real environment used by customers. It contains real user data and real transactions. It must be secure, stable, monitored, and changes are deployed carefully.